

The 13th IJCAR Automated Theorem Proving System Competition – CASC-J13

Geoff Sutcliffe^{a,*}

^a *Department of Computer Science, University of Miami, USA*
E-mail: geoff@tptp.org

Abstract. The CADE ATP System Competition (CASC) is the annual evaluation of fully automatic, classical logic, Automated Theorem Proving (ATP) systems — the world championship for such systems. CASC-J13 was the thirty-first competition in the CASC series. Twenty three ATP systems competed in the various divisions. This paper presents an outline of the competition design and a commented summary of the results.

Keywords: automated theorem proving, competition

Author contributions: With the following caveats, the paper was written by the paper author. The three subsections of Section 7 were written by Nicolas Murzin (FindProof), Olivier Roland (mrs), and Theodore Kim (SUPr). Section 9 was written in collaboration with Julie Cailler and Simon Guilloud.

Statements and Declarations: Ethical considerations - Not applicable. Consent to participate - Not applicable. Consent for publication - Not applicable. Conflicting interest - The author(s) declare no potential conflicts of interest with respect to the research, authorship, and/or publication of this article. Funding statement - Not applicable.

1. Introduction

Automated Theorem Proving (ATP) deals with the task of proving theorems from axioms – the derivation of conclusions that follow inevitably from known facts [1]. The converse task of disproving conjectures is another facet of interest [2, 3]. The axioms and conjecture are written in an appropriately expressive logic, and the solutions (proofs and models) are often similarly written in logic [4]. The CADE ATP System Competition (CASC) [5] is the annual evaluation of fully automatic, classical logic, ATP systems – the world championship for such systems. One purpose of CASC is to provide a public evaluation of the relative capabilities of ATP systems. CASC evaluates the performance of the ATP systems in terms of the number of problems solved, the number of acceptable solutions (proofs and models) output, and the average time taken for problems solved, in the context of a bounded number of eligible problems and specified time limits. Additionally, CASC aims to stimulate ATP research, motivate development and implementation of robust ATP systems that can be easily and usefully deployed in applications, provide an inspiring environment for personal interaction between ATP researchers, and expose ATP systems within and beyond the ATP community.

CASC is held at each CADE (the International Conference on Automated Deduction) and IJCAR (the International Joint Conference on Automated Reasoning) conference – the major forums for the presentation of new research in all aspects of automated deduction. CASC-J13 was held on 27th July 2026, as part of the 13th International Joint Conference on Automated Reasoning (IJCAR 2024)¹, in Lisbon, Portugal. It was the thirty-first competition in the CASC series; see [6] and citations therein, and the CASC web site tptp.org/CASC, for information about

*Corresponding author. E-mail: geoff@tptp.org.

¹CADE was a constituent conference of the 13th IJCAR, hence CASC-“J13”, which in turn was part of the Federated Logic Conference 2026

previous CASCs. The “FLoC Olympic Games” hosted fifteen competitions in various areas of logic and reasoning, which provided the CASC-J13 entrants with opportunities to observe and interact with developers of other types of reasoning tools.

CASC-J13 was organized by Geoff Sutcliffe, and overseen by a panel consisting of Aart Middeldorp, Cláudia Nalon, and Martina Seidl. The CASC panel has a few, but important, responsibilities:

- Advise (by email) on contentious issues that arise before the competition, and that cannot be agreeably resolved by the organizer and entrants.
- With the help of the organizer, adjudicate on the acceptability of the sample proofs and models that are submitted before the competition.
- Supply a seed digit for the random problem selection process.
- If possible, be physically present at the start of the competition, so entrants can see who they are and raise any issues they are concerned about.
- Be contactable during the competition to advise on contentious issues that arise and that cannot be agreeably resolved by the organizer and entrants.
- With the help of the organizer, adjudicate on the acceptability of the winners’ proofs and models.
- If possible, be physically present at the end of the competition for resolving possible open issues, confirming the final system ranking in each division, and assisting in winner announcements.

CASC-J13 was run on computers provided by the StarExec project [7] at the University of Miami. The CASC-J13 web site tptp.org/CASC/J13 provides access to all the resources used before, during, and after the event.

The design and organization of CASC has evolved over the years to a sophisticated state. An outline of the CASC-J13 design and organization is provided here. The details are in [8] and on the CASC-J13 web site. Important changes for CASC-J13 were (for readers already familiar with the general design of CASC):

- The TFA and EPR divisions went on hiatus.
- The FNT division returned from hiatus.
- Inference steps in proofs were expected to list only parents that are not used in the inference.
- Proofs were expected to graft in subproofs from external systems.
- A new performance measure, proof granularity, was recorded and reported.
- The ProoVer competition for proof verifiers was held for the first time in 2026, as a companion competition to CASC-J13.

The CASC rules, specifications, and deadlines are absolute. Only the panel has the right to make exceptions. It is assumed that all entrants have read the documentation related to the competition, and have complied with the competition rules. Non-compliance with the rules can lead to disqualification. A “catch-all” rule is used to deal with any unforeseen circumstances: *No cheating is allowed*. The panel is allowed to disqualify entrants due to unfairness, and to adjust the competition rules in case of misuse.

The rest of this paper is organized as follows: Section 2 describes the competition divisions and the ATP systems that entered the various divisions. Sections 3 and 4 describe the competition infrastructure and the requirements for the ATP systems. Section 5 describes how the systems are evaluated. Section 6 provides a commented summary of the results. Section 7 contains short descriptions of three of the ATP systems. Section 8 discusses the increasingly precise definition and requirements for acceptable proofs in TPTP format, in preparation for CASC-31. Section 9 gives an overview of the companion ProoVer competition, and its ongoing relationship with CASC. Section 10 concludes and discusses plans for future CASCs.

Executive summary: CASC-J13 was the thirty-first annual evaluation of fully automatic, classical logic, Automated Theorem Proving (ATP) systems. CASC-J13 fulfilled its objectives by evaluating the relative capabilities of ATP systems, and stimulating development and interest in ATP. The highlights of CASC-J13 were many new entrants, and the companion ProoVer competition. Despite the continued dominance of the Vampire systems, the stimulating and interactive nature of CASC has kept the developer and user communities engaged. The experience gained from CASC-J13 has prompted further focus on proof production and verification in CASC-31.

A Tense Note: Attentive readers will notice changes between the present and past tenses in this paper. Many parts of CASC are established and stable – they are described in the present tense (the rules are the rules). Aspects that

were particular to CASC-J13 are described in the past tense so that they make sense when reading this after the event.

2. Divisions and Systems

CASC is divided into divisions according to problem and system characteristics, in a coarse version of the TPTP problem library's Specialist Problem Classes (SPCs) [9]. Each division uses problems that have certain logical, language, and syntactic characteristics, so that the ATP systems that compete in a division are, in principle, able to attempt all the problems in the division. Some divisions are further divided into problem categories that make it possible to analyze, at a more fine-grained level, which systems work well for what types of problems. Table 1 catalogs the divisions and problem categories of CASC-J13. The example problems can be viewed online at tptp.org/cgi-bin/SeeTPTP?Category=Problems. Section 3.2 explains what problems are eligible for use in each division and category. The competition rankings are at (only) the division level, as explained in Section 5.

Table 1
Divisions and Problem categories

Division	Problems	Problem categories
THF	Typed (monomorphic) Higher-order Form theorems (axioms with a provable conjecture).	TNE – THF with No Equality Example: NUM738^1 TEQ – THF with Equality Example: SET171^3
FOF	First-Order Form theorems (axioms with a provable conjecture).	FNE – FOF with No Equality. Example: COM003+1 FEQ – FOF with Equality. Example: SEU147+3
FNT	FOF Non-Theorems (axioms with a countersatisfiable conjecture, and satisfiable axioms without a conjecture).	FNN – FNT with No equality. Example: KRS173+1 FNQ – FNT with Equality. Example: MGT033+2
UEQ	Unit Equality theorems in clause normal form (unsatisfiable clause sets).	Example: RNG026-7

ATP systems that cannot run in the competition divisions for any reason, e.g., the system cannot run on the competition computers, or the entrant is an organizer, can be entered into the demonstration division. The demonstration division uses the same problems as the competition divisions, and the entry specifies which competition divisions' problems are to be used. Demonstration division systems can run on the competition computers, or on computers supplied by the entrant. The demonstration division results are presented along with the competition divisions' results, but might not be comparable with those results.

Table 2 lists the 23 ATP systems that competed in CASC-J13, which divisions they entered, and their entrants' information. A division acronym in *italics* indicates the system was in the demonstration division. The division winners of the previous CASC (CASC-30) and the Prover9 1109a system are automatically entered into the demonstration division to provide benchmarks against which progress can be judged. System descriptions are in the competition proceedings [10] and on the CASC-J13 web site.

3. Infrastructure

3.1. Computers

The StarExec Miami computers used for the competition have two octa-core Intel(R) Xeon(R) CPU E5-2620 v4 @ 2.10GHz CPUs, with hyper-threading (two threads per core), 256GiB memory, the Ubuntu 24.04.3 LTS operating system, with Linux kernel 6.8.0-71-generic. There were 30 computers available for CASC-J13, i.e., 60 CPUs. One

Table 2
The ATP systems and entrants

ATP System	Divisions			Entrant (Associates)		Entrant's Affiliation
Connect++ 0.7.2	FOF	FNT		Dr Sean B Holden		University of Cambridge
CSI++ 1.0	FOF		UEQ	Guoyan Zeng (Yang Xu, Baojie Rui, Stephan Schulz, Peiyao Liu, Hailin Guo, Jun Liu, Shuwei Chen)		Xihua University
Drodi 4.1.1	FOF	FNT	UEQ	Oscar Contreras		Amateur Programmer
E 3.5.1	THF	FOF	FNT	UEQ	Stephan Schulz	DHBW Stuttgart
FindProof 0.1		FOF		UEQ	Nik Murzin	Wolfram Institute
FMB4J 0.1			FNT		Michael Rawson (Joe Hall)	University of Southampton
iProver 3.9.4		FOF	FNT	UEQ	Konstantin Korovin	University of Manchester
LEO-II 1.7.0	THF				Alexander Steen (Christoph Benzmlüller)	University of Greifswald
Leo-III 1.9.0	THF				Alexander Steen (Christoph Benzmlüller)	University of Greifswald
LisaST 0.9		FOF			Simon Guilloud	EPFL
Mace4 2026-6A			FNT		Jeff Machado (Larry Lesyna)	Independent Researcher
mrs 0.2.0		FOF		UEQ	Olivier Roland	Independent Researcher
Prover9 1109a		FOF			CASC (Bob Veroff, Bill McCune)	CASC fixed point
Prover9 2026-6A		FOF		UEQ	Jeff Machado (Larry Lesyna)	Independent Researcher
SATResetCoP 1.0		FOF			Martin Fixman (Fredrik Rømming, Sean Holden)	University of Cambridge
SPASS-SCL 0.1.1		FOF			Simon Schwarz (Yasmine Briefs, Lorenz Leutgeb, Christoph Weidenbach, Fabian Wobito)	Max Planck Institute for Informatics
SUPr 1.0		FOF			Teddy Kim (Adam Pease)	Naval Postgraduate School
Twee 2.7				UEQ	Nick Smallbone (Guy Axelrod)	Chalmers University of Technology
Vampire 4.8			FNT		CASC	CASC-29 winner
Vampire 5.0	THF	FOF		UEQ	CASC	CASC-30 winner
Vampire 5.0.1	THF	FOF	FNT	UEQ	Márton Hajdu (Filip Bártek, Ahmed Bhayat, Jonas Bodingbauer, Robin Coutelier, Matthias Hetzenberger, Petra Hozzová, Laura Kovács, Jack McKeown, Merisa Mustajbasic, Axel Polaczek, Jakob Rath, Michael Rawson, Giles Reger, Martin Riener, Martin Suda, Johannes Schoisswohl, Andrei Voronkov, Eva Maria Wagner)	TU Wien
VIP 1.71828459		FOF			Ilies Nokrani (David Delahaye, Clémence Gerardin)	Université Montpellier - LIRMM
Zipperp'n 2.1.9999	THF	FOF			Jasmin Blanchette (Alexander Bentkamp, Simon Cruanes, Petar Vukmirović)	Ludwig-Maximilians-Universität München

ATP system runs on one CPU at a time. StarExec uses Linux's `sched_setaffinity` to restrict each system run to a single CPU, and `setrlimit` to limit memory use to 128 GiB. This separation avoids contention that might affect performance measurements [11, 12]. Systems can use all the cores on the CPU. In CASC-J13 all the demonstration division systems used the competition computers. The StarExec Miami computers used for CASC are the same as are publicly available to the TPTP community, which allows system developers to test and tune their systems in exactly the same environment as is used for the competition.

In CASC-J13 a total of 1618 CPU hours were used in 453 wall clock hours, i.e., 5.9 hours spread over the 60 CPUs, which allowed the competition to run live in one conference day. For the problems that were solved, 185

CPU hours were used in 34 wall clock hours.

3.2. Problems for the TPTP-based Divisions

The problems for CASC are taken from the latest release of the Thousands of Problems for Theorem Provers (TPTP) problem library [13]. For CASC-J13 that was release v9.3.0. The TPTP version used for CASC is released only after the competition has started, so that new problems in the release have not been seen by the entrants. The problems have to meet certain criteria to be eligible for use:

- The TPTP tags problems that are designed specifically to be suited or ill-suited to some ATP system, calculus, or control strategy as *biased*. They are excluded from the competition.
- The problems must be syntactically non-propositional.
- The TPTP uses system performance data in the Thousands of Solutions from Theorem Provers (TSTP) solution library [14] to compute problem difficulty ratings in the range 0.00 (easy) to 1.00 (unsolved) [9]. Problems with ratings in the range 0.21 to 0.99 are eligible. The upper bound of 0.99 excludes problems that probably would not be solved by any of the systems, and thus would not differentiate between the systems. The lower bound of 0.21 was chosen (many years ago, and it has worked successfully) to exclude problems that would be solved by most of the systems and thus not differentiate between the systems. Problems of lesser and greater ratings are made eligible if there are not enough problems with ratings in that range – this was unnecessary in CASC-J13.

Systems can be submitted before the competition so that their performance data is used in computing the problem ratings – problems that are newly solved get a rating less than 1.00 and thus become eligible (until the rating drops below 0.21). The rating calculation also uses performance data from ATP systems that are not entered into the competition, which can produce ratings that make some problems eligible for selection but easy or unsolvable for the systems in the competition. Using problems that are solved by all or none of the competition systems does not affect the competition rankings, has the benefit of placing the systems' performances in the context of the state-of-the-art in ATP, but does reduce the differentiation between the systems in the competition.

In order to ensure that no system receives an advantage or disadvantage due to the specific presentation of the problems in the TPTP problem library, the problems are obfuscated by stripping out all comment lines, including the problem header, randomly reordering the formulae (include directives are left before the formulae, type declarations and definitions are left before the symbols' uses), randomly swapping the arguments of associative connectives, randomly reversing implications, and randomly reversing equalities.

The numbers of problems used in each division and problem category are constrained by the numbers of eligible problems, the numbers of systems entered in the divisions, the number of CPUs available, the time limits, and the time available for running the competition live in one conference day, i.e., in about 6 hours. The numbers of problems used are set within these constraints according to the judgement of the competition organizer. The problems used are randomly selected from the eligible problems based on a seed supplied by the competition panel:

- The selection is constrained so that no division or category contains an excessive number of very similar problems, according to the "very similar problems" (VSP) lists distributed with the TPTP problem library [15]. For each problem category in each division, if the category is going to use N problems and there are L VSP lists that have an intersection of at least $N/(L + 1)$ with the eligible problems for the category, then maximally $N/(L + 1)$ problems are taken from each VSP list.
- In order to combat excessive tuning towards problems that are in the preceding TPTP version, the selection is biased to select problems that are new in the TPTP version used until 50% of the problems in each problem category have been selected or there are no more new problems to select, after which random selection from old and new problems continues. The number of new problems used depends on how many new problems are eligible and the limitation on very similar problems.
- Problems with rating 0.21 to 0.99 are selected before problems with other ratings.

Table 3
Numbers of eligible and used problems

Division	THF		FOF		FNT		UEQ
Category	TNE	TEQ	FNE	FEQ	FNN	FNQ	
Eligible	147	1155	249	4135	91	117	439
Usable	139	439	117	912	64	117	439
Used	100	300	100	300	50	100	400
New	0	31	0	79	0	0	21

Table 3 gives the numbers of eligible problems, the maximal numbers that could be used after taking into account the limitation on very similar problems, and the numbers of problems used in each division and category. As can be seen, the target of 50% new problems was not met, but there were enough new problems in the TEQ, FEQ, and UEQ categories to expose any excessive tuning to the previous TPTP release.

The problems are given to the ATP systems in TPTP format, with `include` directives, in increasing order of TPTP difficulty rating.

3.3. Time Limits

A wall clock time limit is imposed on each system's execution. The minimal time limit is 120 s. The maximal time limit is constrained by the same factors that constrain the numbers of problems that are used, taking into account the phenomenon that ATP systems solve most problems quickly and very few slowly [9, 16]. (This phenomenon is evident in the performance plots from the competition.²) The time limit is chosen value within the range allowed according to the judgement of the organizer, and is announced at the competition. In CASC-J13 a 180 s wall clock time limit was imposed on each system's execution. No CPU time limits are imposed, so it can be advantageous to use all the cores on the CPU.

4. System Registration, Delivery, and Execution

The ATP systems are registered for the competition using an system registration form. Systems can be entered at only the division level, and can be entered into more than one division. Entering many similar variants of a system is deprecated, and entrants may be required to limit the number of variants that they enter. Systems that rely essentially on running other ATP systems without adding value are deprecated, and such systems might be disallowed or moved to the demonstration division. For each system an entrant must be nominated to handle all issues (e.g., installation and execution difficulties) arising before, during, and after the competition. The nominated entrant must formally register for CASC. It is not necessary for entrants to physically attend the competition.

The ATP systems are delivered to the competition organizer as StarExec .tgz installation packages. The installation packages can contain only the components necessary for running the system, i.e., not including source code, etc. The competition organizer installs and tests the packages on StarExec. Source code is delivered separately, under the trusting assumption that the installation package corresponds to the source code.

After the competition all competition division systems' StarExec and source code packages are made available on the competition web site. This allows anyone to use the systems on StarExec, and to examine the source code. Entrants are encouraged to make a public release of their systems after the competition, so that users can enjoy the latest capabilities. An open source license is encouraged, to allow the systems to be freely used, modified, and shared. Many of the StarExec packages include statically linked binaries that provide further portability and longevity of the systems. In the demonstration division the entrant specifies whether or not the source code is placed on the web site.

Execution of the ATP systems is controlled by StarExec. StarExec copies the systems and problems to the compute nodes before starting execution, and timing starts when execution starts. The ATP systems must be fully automatic – they are executed as black boxes, on one problem at a time. Any command line parameters have to be the same for all problems in each division. The ATP systems must be sound, and are tested for soundness before the competition by submitting non-theorems to the systems

²tptp.org/CASC/J13/WWWFiles/ResultsPlots.html

in the THF, FOF, and UEQ divisions, and theorems to the systems in the FNT division. Claiming to have found a proof of a non-theorem or a disproof of a theorem indicates unsoundness. If a system fails the soundness testing it must be repaired by the unsoundness repair deadline or be withdrawn. Three systems were found to be unsound before CASC-J13, and were repaired in time for the competition. The execution of demonstration division systems is supervised by their entrants.

After the competition the systems are tested over the whole TPTP. If a system is found to be unsound, but before the competition report is published, and it cannot be shown that the unsoundness did not manifest itself in the competition, then the system is disqualified. After the competition the systems' solutions are checked. If any of the solutions are found to be unacceptable, but before the competition report is published, then the ranking is revised. All disqualifications and ranking changes are explained in the competition report.

In CASC each ATP system is entered under one name, but it is common for the entrant to deliver multiple installation packages, each specialized to particular divisions in the competition, e.g., separate packages for the FOF and UEQ divisions. Requiring the use of a different package depending on the problem type is a coarse level of non-automation. Users of ATP systems can have different intentions that produce distinct tasks for ATP, the most common of which is theorem proving/unsatisfiability checking vs. countermodel finding/satisfiability checking. In CASC-31 entrants will be restricted to maximally two installation packages, one for each of these two types of task. The ATP systems will be responsible for detecting the problem type within each type of task, and running appropriate components. This will improve the level of automation in ATP systems.

5. System Evaluation

CASC ranks the ATP systems at (only) the division level. For each system, for each problem, four items of data are recorded: whether or not the problem was solved, whether or not a solution (proof or model) was output, the CPU and wall clock times taken to solve the problem and the times taken to output the solution (taken from the time stamps prepended to each line of the system's `stdout` by StarExec's `runsolver` utility), and the maximal number of parents in any inference. The systems are ranked according to the number of problems solved with an acceptable solution output. Ties are broken by the average time taken over problems solved. Trophies are awarded to the competition divisions' winners. A system that is not entered into a division is assumed to perform worse than the entered systems, for that type of problem – wimping out is not an option. In the demonstration division the systems are not ranked, and no trophies are awarded.

The competition panel decides whether or not the systems' solutions are "acceptable". The criteria include:

- Proofs should not use a TPTP language that is more expressive than that of the problem.
- Proofs must be syntactically correct in TPTP format [14]. In CASC-J13 syntax was checked using TPTP4X [18].
- Proofs must be structurally correct:
 - * Annotated formulae must be uniquely named.
 - * Proofs must be acyclic.
 - * Proofs must show only relevant inference steps.
 - * Proofs must have formulae from the problem as leaves, and end at the conjecture (for axiomatic proofs) or `$false` formulae (for proofs by contradiction, e.g., CNF refutations and closed tableau).
 - * Inference steps should list only parents that are used in the inference.
 - * Proofs that make `assumptions` must propagate and eventually discharge the assumptions.

In CASC-J13 structure was checked using GDV [19].

- Translations from one form to another, e.g., FOF to CNF, must be adequately documented.
- Inference steps must be reasonably fine-grained. The maximal number of parents in any inference is considered in this regard.
- Proofs should graft in subproofs from external systems.
- An unsatisfiable set of ground instances of clauses is acceptable for establishing the unsatisfiability of a set of clauses.
- Models must be complete, documenting the domain and symbol maps. The domain and symbol maps may be specified by explicit ground lists (of mappings), or by any clear, terminating algorithm. Saturations are acceptable if their models are a subset of the models of the problem formulae. Use of the TPTP format for interpretations [20] is encouraged.

In addition to the ranking data, other measures are made and presented in the results:

- The *state-of-the-art contribution* (SotAC) quantifies the unique abilities of each system (excluding the previous year's winners that are earlier versions of competing systems). For each problem solved by a system, its SotAC for the problem is the fraction of systems that do not solve the problem. A system's overall SotAC is the average over the problems solved but that are not solved by all the systems.

- The *efficiency* balances the number of problems solved with the time taken. It is the average solution rate (the reciprocal of the time taken to solve it) over the problems solved, multiplied by the fraction of problems solved. Efficiency is computed for both CPU time and wall clock time, to measure how efficiently the systems use one core and multiple cores respectively.
- The *core usage* measures the extent to which the systems take advantage of multiple cores. The raw core usage is the ratio of CPU time to wall clock time used, and the average core usage is over the problems solved with a raw core usage greater than 1.0. The competition ran on octa-core computers, thus the maximal core usage was 8.0. The ability to solve problems quickly before multi-core search is used is also of interest. The number of problems solved with a raw core usage less than 1.0 is noted as an imprecise cutoff for separating proofs that are found in preprocessing on a single core, from proofs found later after multi-core search has started. It is understood that some systems use multiple cores from before 1 s.
- The *granularity* measures the extent to which inference steps combine multiple parent formulae into a single inferred formula. The granularity of a proof is the maximal number of parents in an inference, and the overall granularity is the average over the problems solved with acceptable proof output. Granularity is a new measure in CASC-J13. It was motivated by the very large numbers of parents listed in individual inferences of CASC-30, as discussed in the competition report [6].

6. Results

The result tables below give the number of problems solved with an acceptable solution output, the number of problems solved without considering solution output, the average wall clock time taken over the problems solved, the state-of-the-art contribution, the (micro-)efficiency, the core usage/the number of problems solved with core usage greater than 1.0, the proof granularity, the number of problems uniquely solved, the number of new problems solved, and the number of problems solved in each problem category. In each column the best value in the competition division is **emphasized**. The CASC-30 winner is *emphasized* in the demonstration divisions. Detailed results, including the systems' output files, are available on the CASC-J13 web site.³

The structural checking of proofs in the competition theorem proving divisions was new in CASC-30, and many systems' proofs had structural flaws. In CASC-J13 some developers declared in advance that their systems' proofs would not be acceptable, and their numbers of problems solved with an acceptable solution output is set to "-" rather than 0 in the results tables. Discussion of the progress in proof output from CASC-30 to CASC-J13 is given in Section 8.

6.1. The THF Division

Table 4 summarizes the results of the THF division. Vampire 5.0.1 won the division, but solved less problems than the CASC-30 winner Vampire 5.0, which in CASC-30 had solved less problems than the preceding CASC-J12 winner! The versions of Vampire up to and including 5.0.1 have separate binaries for first-order and higher-order logics. The higher-order components of Vampire are being incrementally merged into the main branch of development, which is clearly producing some growing pains. Zipperposition ran the same code as it has since CASC-29 in 2023, and its proofs that were acceptable in the past are now unacceptable due to the stricter requirements. The flaws in Zipperposition's proofs are quite simple, e.g., many of them have duplicate formulae names, in some cases duplicate formulae, and others give the SZS `status(esa)` when negating the conjecture (it should be `status(cth)`). These are flaws that should be easily fixed, so that Zipperposition is upgraded for CASC-31. The 13 missing proofs from Leo-III are interesting: they come from Leo-III using bracketed quantifiers as terms that are applied, e.g., `((!) @ . . .)`, instead of the combinator equivalents expected in the TPTP language, e.g., `(!! @ . . .)`.

Zipperposition had the highest average time taken, but made the best use of the multiple cores. E solved the most problems before starting to use multiple cores, and had good core usage. Leo-III used multiple cores for all the problems it solved. E solved the most TNE problems, largely because it uniquely solved 19 graph theory (GRA) problems. These problems have been in the TPTP problem library since release v3.6.0 from 2008. E's developer suggested that the progress was simply due to several bugs being fixed in the higher-order code. The new problems were mostly solved by the non-Leo systems.

The granularity figures are not extreme, with the Leos having the highest values. Analysis revealed that their higher granularities came from inferences that were calls to the E system. The developer said: "LEO-II does exactly this: Just list all of the clauses that were sent to E as inference parents. . . . Leo-III does some naive filtering [*of the clauses sent to E*], but it does not filter the parents wrt. actual usage by E. This is ongoing work, but will definitely lead to more realistic/lower number of inference parents on average." The largest contributor to E's granularity was deeply nested `apply_def` steps, in which definitions were sequentially applied in a single inference. Vampire 5.0.1 improved over Vampire 5.0 because the old `avatar_sat_refutation` inferences have been replaced by the CaDiCaL refutation steps. The main motivation for this work was the ongoing proof-checking

³tptp.org/CASC/30/WWWFiles/Results.html

Table 4
THF division results

ATP System	Solns /400	Solved /400	Avg WC	Sot AC	WC μ Eff	Core usage	TNE /100	TEQ /300	Uniq /31	New /31	Gr'ty
Vampire 5.0.1	340	352	6.5	0.32	615	3.9/212	67	285	3	31	10.1
E 3.5.1	323	324	10.7	0.29	593	4.2/129	73	251	23	28	26.4
Leo-III 1.8.0	137	160	14.4	0.07	83	2.5/160	38	122	1	7	53.3
LEO-II 1.7.0	67	67	11.0	0.01	110	0.0/0	26	41	0	0	84.8
Zipperp'n 2.1.9999	0	347	16.7	0.31	334	5.7/268	67	280	1	29	-
Demonstration division											
<i>Vampire 5.0</i>	344	354	6.8	-	576	4.2/238	70	284	3	31	14.8

efforts in Vampire. The developer responsible for this change explained: “it wasn’t so hard with CaDiCaL’s spiffy new Tracer API. You hook your own Tracer subclass into CaDiCaL and it tells you when it learns new clauses.” The developer admitted that although: “the dreaded “avatar sat refutation” is no more. The even-more-dreaded “avatar smt refutation” remains at large.”, which is because that rule is used only for proofs involving arithmetic, as in the TFA division of CASC-30.

The individual problem results (without considering proof output) show that 11 problems were unsolved, 53 problems were solved by all the systems, and 31 problems were solved by only one system. A portfolio (sharing the time between several systems) of Vampire 5.0 and E, with 90 s allocated to each, would solve 383 problems.

6.2. The FOF Division

Table 5 summarizes the results of the FOF division. As in CASC-30, the Vampires dominated the division, but this year there is clear progress from the previous version of the system to the new version. One of the Vampire developers explained: “most of the improvement between 5.0 and 5.0.1 comes from the addition of the neurally guided clause-selection strategies [21] ... Since we solve many hard (previously unsolved) problems with the neurally guided strategies, I think a key was to submit the new version of Vampire early, for the problem rating, so that these problems would become eligible for CASC and could showcase the improvement.” Table 5 and the results plot⁴ show clear grouping of the systems, but the grouping has changed since CASC-30. In CASC-30 there were three groups: the Vampires, a middle group of six systems including E, iProver, Drodi, and Zipperposition, then a lower group of four systems. It was noted in the CASC-30 report that the grouping was less pronounced than in CASC-J12. This year the grouping is even less pronounced, but is still evident: the Vampires are out front as usual, then there is a group of CSI++, E, and iProver, followed by a middle group of Drodi, LisaST, Prover9 2026-6A, and Zipperposition, then a lower group of FindProof, mrs, Prover9—1109a, and SUPr, and finally the bottom group. There are no salient technical features of the systems in each group. The difference seems to be in years of developmental experience. The top groups include systems that have many years of active development and participation in CASC, while the lower groups include newcomers. CASC clearly provides opportunities for system developers to learn from others!

In CASC-30 a key differentiator between the FOF division systems was their abilities to generate acceptable proofs, with a notable difference between the ranking by number of acceptable proofs output and the ranking by the number of problems solved. It is pleasing to see a better correlation between the two rankings in CASC-J13, indicating that lessons were learned from the CASC-30. However, there is still room for further improvement. This year the unacceptable proofs often had simple syntax mistakes that can be easily fixed, e.g., illegal formula names, missing brackets around binary formulae. Some other flaws were deeper, e.g., missing formulae. Vampire 5.0.1 had one proof with a syntax error. LisaST suffered the most, with a mixture of syntax mistakes and deeper flaws. In CASC-31 the requirements for proofs to be considered “acceptable” will be tightened, as explained in Section 8.

A highlight of CASC-J13 was the number of newcomers to the competition. In the FOF division there were four absolute newcomers: FindProof, SUPr, mrs, and VIP. Additionally there were four new variants of existing systems: CSI++, Prover9 2026-6A, LisaST, and SATResetCoP. It was particularly pleasing to see the new release of Prover9, which earned it the “Honourable Mention” medal for CASC in the FLoC Olympic games. The old Prover9 1109a has been the fixed point in the CASC FOF division since CASC-J6 in 2013 (see Section 2). The old version was clearly outperformed by the new version. The developers explained that the main reason for the new version’s strong performance is a sliding window scheduler that rotates through 96 strategies that had been found using machine learning. It is also noteworthy that the new Prover9 outputs proofs in TPTP format.

⁴<https://tptp.org/CASC/J13/WWWFiles/ResultsPlots.html#FOFProblems>

Table 5
FOF division results

ATP System	Solns /400	Solved /400	Avg WC	Sot AC	WC μ Eff	Core usage	FNE /100	FEQ /300	Uniq /48	New /79	Gr'ty
Vampire 5.0.1	380	381	9.5	0.58	309	3.5/346	94	287	45	71	6.0
E 3.5.1	264	264	10.6	0.32	419	4.9/142	75	189	0	55	4.3
CSI++ 1.0	264	264	14.1	0.32	406	4.6/139	76	188	0	55	4.4
iProver 3.9.4	222	250	23.0	0.30	176	4.0/231	64	186	0	54	265.6
Drodi 4.1.1	214	215	20.3	0.24	312	6.0/126	60	155	0	40	10.3
Prover9 2026-6A	139	143	13.0	0.13	208	5.3/98	50	93	0	25	2.0
LisaST 0.9	127	199	18.5	0.21	74	5.1/192	54	145	0	52	2.0
FindProof 0.1	91	91	19.8	0.07	46	6.4/91	26	65	0	18	13.3
SUPr 1.0	73	73	16.7	0.05	92	4.0/54	22	51	0	21	3.7
mrs 0.2.0	71	79	12.6	0.05	106	5.1/59	23	56	0	18	2.5
VIP 1.718	32	32	26.1	0.01	34	0.0/0	13	19	0	8	2.0
Connect++ 0.7.2	24	64	60.0	0.04	22	0.0/0	27	37	0	13	2.0
SATResetCoP 1.0	20	20	70.5	0.01	3	0.0/0	3A	17	0	9	456.0
SPASS-SCL 0.1.1	14	14	21.9	0.01	12	0.0/0	14	0	0	0	4.7
Zipperp'n 2.1.9999	0	192	19.6	0.18	196	4.9/143	46	146	0	47	-
Demonstration division											
<i>Vampire 5.0</i>	328	328	8.1	-	538	3.9/236	80	248	3	70	13.9
Prover9 1109a	-	92	14.7	-	134	0.0/0	23	69	0	22	2.0

Vampire 5.0.1's high SotAC correlates to its high number of unique solutions. E's higher efficiency comes from solving most problems very quickly, with relatively fewer being solved slowly compared to the Vampires. This can be seen in the results plot, with E having a sharper upwards swing than the Vampires. mrs also had a high efficiency relative to its ranking. FindProof had the highest core usage, using multiple cores in all the problems it solved. LisaST used multiple cores for most of the problems it solved, which is the intention of the system design as the developer explained: "there are 8 built-in strategies, and it always runs the 8 in parallel". Drodi also had high core usage, and solved 89 problems with a raw core usage less than 1.0. The developer of Drodi provided analysis of the first five and last five problems solved in less than 1 s, revealing that eight of those 10 proofs were found during multi-core search. He explained: "These results suggest that more than one core is used in most of the problems solved in under 1 second and in some cases even a significant number of cores are used." ASKED OSCAR FOR MORE. Several of the systems, in particular Connect++, SATResetCoP, and VIP, had high average wall clock times and no core usage. The expected benefit of using multiple cores to reduce wall clock time taken, as noted in Section 3.3, is evident here.

The granularity values are reasonable for most of the systems. The best came from Connect++, LisaST, Prover9 2026-6A, and VIP, which all had a granularity of 2.0. That means those systems' inferences had an average maximum of two parents, making the proofs as fine-grained as can be expected. At the other end of the scale, iProver and SATResetCoP had high granularity values. iProver had eight proofs with a granularity of over 1000, three of which were over 10000, with the maximum being 12651. As was the case in CASC-30, iProver's use of Z3 finish refutations is the common culprit, seen in iProver's inference rule `smt_impl_just`. SATResetCoP's proof structure always contains a formulae that is the conjunction of the axioms and negated conjecture, i.e., the granularity of the proof is the size of the problem. The problem `CSR058+3` has 8006 formulae, and thus the granularity of SATResetCoP's proof is 8006. Outliers such as these contributed to SATResetCoP's higher granularity value. The developer said (about these formulae): "This is generated by the certificate generator, and not by SATResetCoP instead. It might be not crucial for the final output."

The performances in the two problem categories are well aligned with the overall performance, except for LisaST's stronger performance on problems with equality, and SPASS-SCL's zero performance on problems with equality. The new problems provided consistent challenges for the systems, with about 20% of the problems being new problems, and for most of the systems about 20% of the problems solved were new problems. The exception is Prover9 2026-6A, for which 17.5% of the problems solved were new problems. Careful analysis by the developers concluded that this is: "a non-significant point estimate on a 79-problem sample, not evidence of tuning". Only 2 of the 45 problems uniquely solved by Vampire 5.0.1 were new problems.

The individual problem results (without considering proof output) show that 14 problems were unsolved, no problems were solved by all the systems, and 48 problems were solved by only one system. Between them, the two Vampires solved a superset of the union of the problems solved by the other systems.

6.3. The FNT Division

The FNT division return to CASC-J13 after two years on hiatus. The winner from CASC-29 in 2003 was entered as the previous winner. Table 6 summarizes the results. Vampire 5.0.1 came out ahead of the other systems, by a large margin, surpassing Vampire 4.8 that had come out way ahead in CASC-29. The key to the Vampires' success was strategy scheduling that tries to saturate the clause set and alternatively find a finite model. The only other system that produced two types of models was iProver, which output saturations and models expressed in a term algebra. The ratios of saturations to (finite) models output was as follows: Vampire 5.0.1 - 59:84, Vampire 4.8 - 36:91, iProver - 6:65, FMB4J 0:51, Mace4 0:44, E - 42:0, and Drodi - 41:0.

In contrast to the three theorem proving divisions, there was no checking of the models output in the FNT division. Post-competition analysis showed that all the competition systems' models passed TPTP syntax checking, except for iProver in which some models had duplicate formula names and unquantified variables – these simple errors are easily rectified. Some of Vampire 4.8's models also had simple errors, but that's history. For one problem Mace4 found a model but did not complete the model output by the time limit. Syntax checking of models will be added in CASC-31, as explained in Section 8.

Table 6
FNT division results

ATP System	Solns /150	Solved /150	Avg WC	Sot AC	WC μ Eff	Core usage	FNN /50	FNQ /100	Uniq /16	New /10
Vampire 5.0.1	143	143	11.9	0.58	434	4.7/87	45	98	14	10
iProver 3.9.4	71	71	25.5	0.19	176	3.3/53	15	56	0	8
FMB4J 0.1	51	51	9.3	0.11	241	1.3/33	8	43	0	7
Mace4 2026-6A	44	45	9.0	0.10	229	4.2/12	8	37	0	8
E 3.5.1	42	42	9.4	0.12	151	4.3/22	17	25	0	2
Drodi 4.1.1	41	41	6.1	0.09	175	4.9/17	7	34	0	2
Connect++ 0.7.2	-	11	0.4	0.01	73	0.0/0	3	8	0	0
Demonstration division										
<i>Vampire 4.8</i>	127	131	17.4	-	411	5.6/82	45	86	2	10

Vampire 5.0.1 dominated the division across all performance measures. Drodi stood out as having a low average CPU time coupled with a high core usage – the insights from Section 6.2 apply again here. The problem category rankings reflect the overall ranking except for E's better performance on FNN problems. This is different from CASC-29 in which E 3.1 performed relatively better on FNQ problems. The developer of E ascribed the change to: "just random luck" of the problem selection.

The individual problem results show that 4 problems were unsolved, 2 problems were solved by all the systems, and 16 problems were solved by only one system. As is the case in the FOF division, between them, the two Vampires solved a superset of union of the problems solved by the other systems.

6.4. The UEQ Division

Table 7 summarizes the results of the UEQ division. Vampire 5.0.1 won, and as in the FOF division showed significant improvement over Vampire 5.0, for the same reasons. The goal directed technique pioneered by Twee [22] is one of the techniques that led to Vampire 4.9 winning the UEQ division of CASC-J12 in 2024, and remains in Vampire 5.0.1. Its value is well appreciated by the Vampire developers, one of whom said: "tgt-ground [*the goal directed technique*] is definitely worth having. We are actually considering baking it into the default strategy." E also tried to adopt the idea, but sadly, as the developer explained: "my hack for CASC was in vain ;-). I carefully added the goal-directed transformation strategy to every search schedule, but ... the preprocessing schedules are now separate data structures, and the transformation was only relevant in preprocessing".

FindProof, mrs, and Prover9 failed to output acceptable proofs for all problems solved. FindProof simply did not output any proof for 18 problems, and mrs output one proof with syntax errors. For Prover9, as the developers put it when fixing the output for their next release: "The bug is an amusing one. Prover9 still registers Otter's legacy infix $\vee$ (disjunction) as a parse rule, so when a TPTP problem legitimately uses $\vee$ as a binary functor, the printer rendered the legacy in infix notation."

The average times taken and SotAC values are roughly aligned with the division ranking. CSI++ had a high efficiency because it solved many problems quickly, typically through its use of E. As in the FOF and FNT divisions, Drodi and FindProof had high core usage. Vampire 5.0.1 and Prover9 performed best on the new problems, with Prover9 showing the least bias towards old problems. The granularity values are reasonably low, with Prover9 having an optimal value.

Table 7
UEQ division results

ATP System	Solns /400	Solved /400	Avg WC	Sot AC	WC μ Eff	Core usage	Uniq /30	New /21	Gr'ty
Vampire 5.0.1	370	370	11.8	0.40	222	4.1/351	17	12	7.9
Twee 2.7	286	286	22.4	0.24	225	6.4/273	2	7	2.0
CSI++ 1.0	258	258	21.5	0.19	261	5.0/223	1	9	10.4
E 3.5.1	255	255	20.4	0.19	244	5.3/226	1	8	12.3
FindProof 0.1	251	269	25.3	0.21	118	6.5/269	1	4	2.3
Prover9 2026-6A	207	208	21.4	0.15	202	6.2/179	4	10	2.0
iProver 3.9.4	198	198	37.9	0.12	73	5.1/189	1	7	10.5
Drodi 4.1.1	163	163	37.2	0.09	127	6.7/143	0	3	4.2
mrs 0.2.0	75	76	36.8	0.02	33	5.8/71	0	2	3.8
Demonstration division									
<i>Vampire 5.0</i>	322	322	16.6	-	295	5.2/310	3	13	8.7

The individual problem results (without considering proof output) show that 3 problems were unsolved, 43 problems were solved by all the systems, and 30 problems were solved by only one system. A portfolio does not help much, the best being a combination of Vampire 5.0.1 and CSI++ 1.0 with 90 s allocated to each, which would solve 373 problems.

7. System Descriptions

Each year the competition report features short system descriptions of systems that stand out in some way. For CASC-J13 the salient systems are the newcomers that performed relatively well (as newcomers) in the FOF division: FindProof 0.1, SUPr 1.0, and mrs 0.2.0. These short descriptions of the systems were written by their entrants.

FindProof [23] is an equational theorem prover for unit-equality problems, based on unfailing (ordered) Knuth-Bendix completion [24]. It continues the Waldmeister line of provers [25]: the completion loop, critical-pair selection, queue management, and normalisation. On algebraic problems the default configuration reproduces Waldmeister's critical-pair selection sequence exactly. The axioms are saturated towards a convergent rewrite system while the goals are kept normalised. Equations that cannot be oriented by the reduction ordering are used unfailingly, so that both faces are superposed, which preserves refutational completeness. The Knuth-Bendix ordering and the lexicographic path ordering are supported, with symbol precedences generated automatically. Redundant critical pairs are discarded by ground-joinability testing, the connectedness criterion, forward and backward subsumption and demodulation, and right-hand-side interreduction. Proofs are reconstructed from the recorded completion trace, the closing rewrite chain re-derived against the final rule set and rendered as a refutation. The search is ordered by a weighted critical-pair selection heuristic over a priority queue, with a Waldmeister-style selection ratio that interleaves a first-in-first-out pick at a fixed cadence, so that completeness is retained. The full strategy surface, some fifty-five switches covering the reduction ordering, critical-pair weighting, weight limits, redundancy criteria, goal treatment, and queue management, is exposed as command-line options, and named configurations modelled on the published defaults of Waldmeister, Twee, and Vampire are provided. In the competition a fixed schedule of such configurations was run, each in its own child process under a slice of the wall-clock limit, stopping at the first proof. FindProof is a single self-contained C program with no external dependencies.

SUPr [26] is a saturation-based theorem prover built as a native component of the Sigma knowledge engineering environment (Rust implementation – SigmaKEE-rs) [27]. SUPr is optimized to reason directly over the Suggested Upper Merged Ontology (SUMO) [28], with additional support to solve standalone TPTP problems. The core search procedure is a given-clause saturation loop implementing ordered resolution and superposition with selective literal selection, forward and backward demodulation, and a Knuth-Bendix reduction ordering. Clauses and their constituent terms are represented in a content-addressed, hash-consed arena. This content based addressing scheme is structured to allow batched, hardware accelerated unification via binary arithmetic. As SUPr is built specifically for reasoning over SUMO, SUPr's proving strategy depends on heavy caching of supporting evidence for new axioms at the time of ingest, and use-case-specific semantic decoding. Novel pre-caching of SUMO Inference Engine [29] (SInE) occurrences and trigger indices allows for fast per-problem axiom selection. Axioms are additionally ranked by structural relevance to the conjecture [30], and those that rank highly but were dropped by SInE are rescued and reintroduced to the problem. Search-loop tuning is organized as a small number of strategies running as concurrent threads that race for a solution. Threads share a single clause stack that remains unconflicted due to the atomicity of the content-based hashing scheme.

mrs [31] is an automated theorem prover for first-order logic with equality. It implements the superposition calculus [32] in a given-clause loop, with ordered resolution, factoring, equality resolution, equality factoring, and paramodulation, all oriented by KBO or LPO with rarity-based symbol precedence. Non-Horn clauses are split via AVATAR [33] using the CaDiCaL [34] SAT solver. **mrs** is written in Rust. The search runs a parallel portfolio, with each strategy maintaining its own clause set. A central design feature is proof-preserving cross-strategy sharing: workers exchange selected positive unit equalities with complete ancestor chains, remap those chains into the receiving proof store, and use only the equality heads in subsequent search. Rust's ownership and thread-safety checks make the synchronized shared pool explicit and race-safe. The portfolio for each category division (FNE, FEQ, UEQ) was selected by running all 15 base strategies solo and using greedy set-cover analysis to choose an eight-strategy portfolio. Local analysis identified clause generation and redundancy, rather than raw iteration speed, as the main bottleneck. The competition's objective scores and rapid feedback loop made profiling, debugging, and heuristic experiments focused, repeatable, and fun, directly shaping subsequent work on inference restrictions, resource controls, and proof-preserving sharing. CASC pushed **mrs** to fix real bugs and close gaps that show up only at scale.

8. Acceptable Proofs

In the commentary of the results in the THF, FOF, and UEQ divisions (Sections 6.1, 6.2, and 6.4, the extent to which the systems output acceptable proofs is discussed. For newcomers to CASC imperfect proof output might be forgiven. For the old timers, the increasing emphasis on proof production since CASC-J12 should be having a positive effect. This section examines the requirements for acceptable proofs for the last three CASCs – CASC-J12, CASC-30, and CASC-J13, and evaluates the progress made between CASC-30 and CASC-J13 for systems that were entered in both.

8.1. Requirements

Prior to CASC-30 the opinion of the CASC panel was the only adjudication of whether systems' proofs were acceptable. These guidelines were set for CASC-J12 [10]:

1. Proofs must be complete, starting at formulae from the problem, and ending at the conjecture (for axiomatic proofs) or a false formula (for proofs by contradiction, e.g., CNF refutations).
2. For proofs that use translations from one form to another, e.g., translation of FOF problems to CNF, the translations must be adequately documented.
3. Proofs must show only relevant inference steps.
4. Inference steps must document the parent formulae, the inference rule used, and the inferred formula.
5. Inference steps must be reasonably fine-grained.

The CASC-J12 report [35] noted the benefits of moving towards a “verified track” in CASC. An incremental approach was adopted, and the following requirements were added in CASC-30:

6. Proofs must be in the TPTP format.
7. Annotated formulae in a proof must be uniquely named.
8. Proofs must be structurally correct, e.g., derivations must be acyclic.
9. Proofs that negate the conjecture must give the resultant formula the role `negated_conjecture`, and annotate the step as `status(cth)`.
10. Proofs that introduce new symbols in definitions must provide an `introduced()` record to document the new symbol.

The TPTP4X parser and GDV proof verifier were made available to the systems' developers so that they could check their proofs in using the same tools as were used for the competition evaluation (see Section 5). Note that there was no requirement for logical correctness, and no proof verification was done.

The impact was interesting, and the data showed that the ATP systems still needed to improve their proofs [6]. A particular phenomenon observed was that many of the ATP systems' proofs included “inference steps” that were calls to SAT/ATP/SMT systems that inferred `$false` to end the refutation. These final calls to external systems were typically reported with all the clauses sent to the external system listed as parents – the most extreme case listed 72367 parent clauses that were sent to the external Z3 system! A limited analysis revealed that typically only a subset of the listed parents were used by the SAT/ATP/SMT system to infer `$false`.

To encourage developers to improve their systems' proofs, and avoid the second weakness, the requirements for acceptable proofs were refined in CASC-J13:

11. Inference steps must list exactly those parents that are used in the inference. The granularity measure was added to the results to track the maximal numbers of parents in the inference steps of proofs.
12. Proofs that make assumptions must propagate and eventually discharge the assumptions.

8.2. Progress from CASC-30 to CASC-J13

Table 8 provides data comparing the progress in acceptable proof production between CASC-30 and CASC-J13, for systems that had versions entered in both competitions. Loss measures what percentage of the problems solved did not have acceptable proof output:

$$Loss \% = \frac{Solved - Solns}{Solved}$$

A reduction in Loss, indicated by a negative Δ Loss, directly indicates improvement.

The Progress column measures that improvement in the context of the numbers of problems solved and the numbers of problems in the divisions:

$$Progress = \begin{cases} \text{if } 0\% \text{ Loss both years,} & CASC-30\left(\frac{Solns}{Problems}\right) - CASC-J13\left(\frac{Solns}{Problems}\right) \\ \text{otherwise,} & CASC-30\left(\frac{Solved - Solns}{Problems}\right) - CASC-J13\left(\frac{Solved - Solns}{Problems}\right) \end{cases}$$

The Progress values for systems that had 0% loss in both years, when $Solved = Solns$ and indicated by a '-' in the Δ Loss column, are not directly comparable with the others, but do measure progress in acceptable proof production.

There are very definite signs of progress, particularly in the FOF division. Drodi's improvement is salient, with iProver also having significant improvement (again in the UEQ division). E is noteworthy for having consistently minimal loss, coupled with improved performance. As an overall measure, in CASC-30 459 of the 3961 proofs from the THF, FOF, and UEQ divisions, i.e. 11.5%, had syntax or structural flaws. The situation improved in CASC-J13 with only 200 of 5065 proofs, i.e., 3.9%, having those types of flaws.

Table 8
Progress in acceptable proof output

Division System	CASC-30 Version	Problems Solved	Solns	Loss	CASC-J13 Version	Problems Solved	Solns	Loss	Δ Loss	Progress
THF		500				400				
E	3.3.0	395	375	5.06%	3.5.1	324	323	0.31%	-4.75%	3.75%
Vampire	5.0	449	439	2.23%	5.0.1	352	340	3.41%	1.18%	-1.00%
Leo-III	1.7.19	242	241	0.41%	1.8.0	160	137	14.38%	13.96%	-5.55%
FOF		500				400				
Drodi	4.1.0	325	210	35.38%	4.1.1	215	214	0.47%	-34.92%	22.75%
iProver	3.9.3	367	278	24.25%	3.9.3	250	222	11.20%	-13.05%	10.80%
Connect++	0.6.1	102	0	100.00%	0.7.2	64	24	62.50%	-37.50%	10.40%
E	3.3.0	364	364	0.00%	3.5.1	264	264	0.00%	-	6.80%
SPASS-SCL	0.1	11	0	100.00%	0.1.1	14	14	0.00%	-100.00%	2.20%
Vampire	5.0	455	455	0.00%	5.0.1	381	380	0.26%	0.26%	-0.25%
UEQ		300				400				
iProver	3.9.3	205	62	69.76%	3.9.3	198	198	0.00%	-69.76%	47.67%
Twee	2.6.0	258	258	0.00%	2.7	286	286	0.00%	-	14.50%
E	3.3.0	222	222	0.00%	3.5.1	255	255	0.00%	-	10.25%
Vampire	5.0	263	263	0.00%	5.0.1	370	370	0.00%	-	-4.83%
Drodi	4.1.0	121	121	0.00%	4.1.1	163	163	0.00%	-	-0.42%

A comparison of the granularity values between CASC-30 and CASC-J13 would be “unfair”, because the measure was not part of the CASC-30 design, and thus developers had no motivation to minimise granularity. The granularity values reported in Sections 6.1, 6.2, 6.3, and 6.4 provide some indication of the situation in CASC-J13. As a counter point to the maximal value of 72367 clauses sent to Z3 in CASC-30, the maximal value in CASC-J13 was 12651.

8.3. The Future

Analysis after CASC-J13 revealed that many proofs had ill-formed documentation in their `introduced` and `inference()` records. The CASC-J13 panel ruled that the documentation of the required format for these records was inadequate, and thus those flawed proofs were counted. The documentation has been improved for these two points and more, and more precise requirements have become possible. These are given in requirements 12', 13', and 14 below.

Discussions during the IJCAR-13 conference that hosted CASC-J13 led to the conclusion that requirement 11 is unrealistic because precisely extracting the set of parents that were used by a SAT/ATP/SMT system might not be possible in all cases. From the organizational side, checking conformance to the requirement might not be possible in all cases. It became clear that the systems can only “aim to” comply, leading to a change to the requirement, specified in requirement 11' below. Even with a minimization of the parents list in inference steps, it is debatable whether a successful call to a SAT/ATP/SMT system is really an “inference step”. The SAT/ATP/SMT systems typically build a proof of their own, and that proof structure is often not shown in the ATP system's proof. It would be preferable for the SAT/ATP/SMT system's proof to be exposed by being grafted onto the ATP system's proof (as done in Vampire 5.0.1's use of CaDiCaL - see Section 6.1). Grafting on the proofs of subsystems will be encouraged in CASC-31, specified in requirement 15 below.

In addition to these reactive changes to observations from CASC-J13, two further requirements will be added. Firstly, users of ATP systems can reasonably expect proofs to not use TPTP language features that are not supported by the language used for the problem. For example, proofs for FOF problems should not use THF higher-order features, and if that restriction is observed then the proofs can be expected to be written in FOF. That that end requirement 9 below will be added. Secondly, as another incremental step towards a verified track in CASC, leaves of proofs will be required to directly copied from the problem, modulo variable renaming. That that end requirement 10 below will be added. This is in agreement with the requirements of the ProoVer competition described in Section 9.

The requirements for CASC-31 will include:

9. Proofs should not use a TPTP language that is more expressive than that of the problem.
10. Leaf annotated formulae of proofs must be alpha-equivalent to the corresponding named annotated formula in the problem.
- 11'. Inference steps must aim to list exactly those parents that are used in the inference. Inference steps may list parents that are not necessary for the inference.⁵
- 12'. Proofs that negate the conjecture must correctly annotate the step as `status(cth)` and have a single parent with the role `conjecture`.
- 13'. New symbols introduced in derivations must be documented in a `new_symbols()` record. The new symbols listed in a `new_symbols()` record must be unique and globally new.
14. Requirements for the `introduced()` records of symbol definitions, introduced tautologies, and theory axioms will be provided. Requirements for the `inference()` records of Skolemization steps will be provided.
15. Proofs should graft in subproofs from external systems.

The BNF-based parser [17], the TPTP4X parser, and an improved version of GDV will be made available to the systems' developers so that they can check their proofs. This can be done online in SystemOnTSTP.⁶ These more rigorous requirements for proofs in CASC resonate with the ProoVer proof verifier competition that was held as a companion competition to CASC-J13. ProoVer is described in Section 9.

8.4. Acceptable Models

Up to and including CASC-J13, the CASC panel was the only adjudication of whether systems' models were acceptable. The following guidelines were set for CASC-J13 (see Section 5):

⁵For example, in the inference of $p(a)$ from the parents $p(a)$ and $a = a$, the equality is not necessary but might be used in an (unnecessary) equality substitution step, but in contrast $p(b)$ would probably not be used as a parent in that inference step.

⁶tptp.org/cgi-bin/SystemOnTSTP

Models must be complete, documenting the domain and symbol maps. The domain and symbol maps may be specified by explicit ground lists (of mappings), or by any clear, terminating algorithm. Saturations are acceptable if their models are a subset of the models of the problem formulae. Use of the TPTP format for interpretations is encouraged.

For CASC-31 the requirements will be made more precise:

1. Models must be written as annotated formulae in TPTP syntax.
2. Annotated formulae in a model must be uniquely named.
3. Models must be complete, documenting the domain and symbol maps.
4. The domain and symbol maps may be specified by explicit ground lists (of mappings), or by any clear, terminating algorithm.
5. Models must show all and only relevant information.
6. Use of the TPTP format for interpretations is encouraged.

Models will syntax checked using the BNF-based parser and the TPTP4X parser.

9. The ProoVer Competition

The Proof Verification Competition (ProoVer) is a new competition, evaluating proof verifiers, i.e., systems that verify whether a proof is correct wrt its conjecture. ProoVer 2026 was the first edition, held as a companion competition to CASC-J13, sharing its computers and much of its organizational machinery. ProoVer 2026 was organized by Julie Cailler and Simon Guilloud, and overseen by the same panel as CASC-J13: Cláudia Nalon, Aart Middeldorp, and Martina Seidl. An overview of the competition is given here. Full details of the competition design and the entrant systems are given in the ProoVer 2026 proceedings [37].

A ProoVer 2026 entrant is a proof verifier that is given an untyped first-order (FOF) TPTP problem and a corresponding refutation proof [38], and decides whether or not the refutation proof is valid. A refutation proof consists of a sequence of syntactically correct TSTP inference steps ending at `$false`. Most inference steps could be discharged by an external ATP system, but the verifiers had to verify a small set of specified rules internally – Skolemization, negation of the conjecture, and correspondence between the problem file and the proof file. A wall clock time limit of 60 s was imposed on each verification attempt. The systems had to report their decision using the SZS ontology, one of VerifiedGood (VSG), VerifiedBad (VSB), or Unknown (UNK). Any other result, e.g., a timeout, was treated as UNK.

The systems were ranked according to the following grading scheme:

- Identifying a bad proof as bad = +2
- Identifying a good proof as good = +1
- Unknown = 0
- Identifying a good proof as bad = -1
- Identifying a bad proof as good = -10

The problem set originally had 50 good proofs and 50 bad (aka “evil”) ones that had invalid inferences or other bad features. However, 9 proofs were later removed as ambiguous or non-compliant, leaving 44 good and 47 bad proofs that were used for the ranking. The possible score therefore ranged from 138 to -514.

Ten proof checkers competed: eight in the ranked regular division, and two in the unranked demonstration division (due to the need for internet access). Table 9 lists the entrants and the results. GAPT 2.20 won the competition with 111 points. Since at least one student was involved in the development of GAPT 2.20, it also won the *ProoVer 2026 Best Student Proof Checker* award. VaLeaDate 0.1 won the *ProoVer 2026 Safest Proof Checker* award: it tied with Nörgler for the most Bad proofs correctly identified (46 out of 47), but it did so without any VerifiedGood verdicts on a Bad proof. On the Good proofs side, PyCheck 0.1 performed best, correctly identifying 37 Good proofs against only two misclassified as Bad. The ProoVer 2026 problem set is available online, together with an explanation of each problem. ProoVer’s first edition led to multiple discussions, serving the broader goal of improving the TSTP specification.

10. Conclusion

CASC-J13 was the thirty-first annual evaluation of fully automatic, classical logic, Automated Theorem Proving (ATP) systems. CASC-J13 fulfilled its objectives by evaluating the relative capabilities of ATP systems, and stimulating development and interest in ATP. The highlight of CASC-J13 was the large number of newcomers who provided counterpoints for the systems that

Table 9
ProoVer 2026 summary

		44 Good proofs				47 Bad proofs				Score
Verifier	Entrants	VSG	VSb	UNK	/44	VSb	VSG	UNK	/94	/138
GAPT 2.20	Fabian Achammer & Martin Riener	34	5	5	29	41	0	6	82	111
VaLeaDate 0.1	Jonas Bodinbauer & Laura Kovacs	24	18	2	6	46	0	1	92	98
Nörgler 1.1	Alexander Steen et al.	27	16	1	11	46	1	0	82	93
ProofCheck 1.0	Jeff Machado & Larry Lesyna	31	10	3	21	42	3	2	54	75
ProofGuard 1.0	Matthew Farah & Alicia Marinache	30	11	3	19	40	5	2	30	49
mrs-proover 0.2.0	Olivier Roland	31	12	1	19	39	5	3	28	47
PyCheck 0.1	Stephan Schulz	37	2	5	35	28	5	14	6	41
CheckProof 0.1	Nik Murzin	28	11	5	17	25	17	5	-120	-103
<i>Demonstration division</i>										
GDV 2.0	Geoff Sutcliffe	35	5	4	30	8	5	34	-34	-4
GDV-LP 2.0	Frédéric Blanqui & Geoff Sutcliffe	33	5	6	28	8	5	34	-34	-6

have been entered for many years. The lowlight was the number of syntactically and structurally flawed proofs, which hopefully will motivate developers to pay more attention to this aspect of their ATP systems. Despite the continued dominance of the Vampire systems, the developer and user communities have remained engaged with CASC. A key to this is CASC's stimulating and interactive nature: a live competition run during one day of the conference that provides opportunities for developers, users, and observers to interact.

While the design of CASC is mature and stable, each year's experiences lead to ideas for changes and improvements. Changes planned for CASC-31 are:

- Only two installation packages will be allowed for each entry – one for theorem proving/unsatisfiability checking, and one for countermodel finding/satisfiability checking. The organizer will provide template `starexec_run` packages to smooth out the creation and submission of working installation packages.
- All solutions – proofs and models – must be in TPTP syntax, error-free according to the BNF-based and TPTP4X parsers.
- Proof checking will be extended as described in Section 8.3.
- No sample proofs will be requested, to avoid any mistaken belief that one sample proof approved by the panel implies that all proofs will be acceptable.

As always, the ongoing success and utility of CASC depends on ongoing contributions of problems to the TPTP. The automated reasoning community is encouraged to continue making contributions of all types of problems.

References

- [1] A. Robinson and A. Voronkov, *Handbook of Automated Reasoning*, Elsevier Science, Amsterdam, The Netherlands, 2001.
- [2] K. Claessen and N. Sörensson, New Techniques that Improve MACE-style Finite Model Finding, in: *Proceedings of the CADE-19 Workshop: Model Computation - Principles, Algorithms, Applications*, P. Baumgartner and C. Fermueller, eds, 2003.
- [3] J. Blanchette and T. Nipkow, Nitpick: A Counterexample Generator for Higher-Order Logic Based on a Relational Model Finder, in: *Proceedings of the 1st International Conference on Interactive Theorem Proving*, M. Kaufmann and L. Paulson, eds, Lecture Notes in Computer Science, Springer, Berlin, Germany, 2010, pp. 131–146.
- [4] G. Sutcliffe, The Logic Languages of the TPTP World, *Logic Journal of the IGPL* **31**(6) (2023), 1153–1169.
- [5] G. Sutcliffe, The CADE ATP System Competition - CASC, *AI Magazine* **37**(2) (2016), 99–101.
- [6] G. Sutcliffe, The CADE-30 Automated Theorem Proving System Competition - CASC-30, *AI Communications* (2026), To appear. doi:10.1177/30504554261463374.
- [7] A. Stump, G. Sutcliffe and C. Tinelli, StarExec: a Cross-Community Infrastructure for Logic Solving, in: *Proceedings of the 7th International Joint Conference on Automated Reasoning*, S. Demri, D. Kapur and C. Weidenbach, eds, Lecture Notes in Artificial Intelligence, 2014, pp. 367–373.
- [8] G. Sutcliffe, *Proceedings of the 13th IJCAR ATP System Competition*, Online, 2026, <http://tptp.org/CASC/J13/Proceedings.pdf>.
- [9] G. Sutcliffe and C.B. Suttner, Evaluating General Purpose Automated Theorem Proving Systems, *Artificial Intelligence* **131**(1–2) (2001), 39–54. doi:10.1016/S0004-3702(01)00113-8.
- [10] G. Sutcliffe, *Proceedings of the 12th IJCAR ATP System Competition*, Online, 2024, <http://tptp.org/CASC/J12/Proceedings.pdf>.

- [11] D. Beyer, S. Löwe and P. Wendler, Reliable Benchmarking: Requirements and Solutions, *International Journal on Software Tools for Technology Transfer* **21** (2019), 1–29. doi:10.1007/s10009-017-0469-y.
- [12] J. Fichte, T. Geibinger, M. Hecher and M. Schlögel, Parallel Empirical Evaluations: Resilience despite Concurrency, in: *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, M. Wooldridge, J. Dy and S. Natarajan, eds, Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, AAAI Press, Palo Alto, USA, 2024, pp. 8004–8012.
- [13] G. Sutcliffe, The TPTP Problem Library and Associated Infrastructure. From CNF to TH0, TPTP v6.4.0, *Journal of Automated Reasoning* **59**(4) (2017), 483–502. doi:10.1007/s10817-017-9407-7.
- [14] G. Sutcliffe, TPTP, TSTP, CASC, etc., in: *Proceedings of the 2nd International Symposium on Computer Science in Russia*, V. Diekert, M. Volkov and A. Voronkov, eds, Lecture Notes in Computer Science, Springer, Berlin, Germany, 2007, pp. 6–22.
- [15] G. Sutcliffe, The CADE-16 ATP System Competition, *Journal of Automated Reasoning* **24**(3) (2000), 371–396. doi:10.1023/A:1006393501098.
- [16] G. Sutcliffe, Stepping Stones in the TPTP World, in: *Proceedings of the 12th International Joint Conference on Automated Reasoning*, C. Benz Müller, M. Heule and R. Schmidt, eds, Lecture Notes in Artificial Intelligence, 2024, pp. 30–50.
- [17] A. Van Gelder and G. Sutcliffe, Extending the TPTP Language to Higher-Order Logic with Automated Parser Generation, in: *Proceedings of the 3rd International Joint Conference on Automated Reasoning*, U. Furbach and N. Shankar, eds, Lecture Notes in Artificial Intelligence, Springer, Berlin, Germany, 2006, pp. 156–161.
- [18] G. Sutcliffe, The TPTP World - Infrastructure for Automated Reasoning, in: *Proceedings of the 16th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning*, E. Clarke and A. Voronkov, eds, Lecture Notes in Artificial Intelligence, Springer, Berlin, Germany, 2010, pp. 1–12.
- [19] G. Sutcliffe and D. Belfiore, Semantic Derivation Verification, in: *Proceedings of the 18th International FLAIRS Conference*, I. Russell and Z. Markov, eds, AAAI Press, Palo Alto, USA, 2005, pp. 641–646.
- [20] G. Sutcliffe, A. Steen, P. Fontaine and L. Kondylidou, The TPTP Format for Interpretations, in: *Proceedings of the 10th Workshop on Practical Aspects of Automated Reasoning*, C. Nalon, M. Hajdu and M. Suda, eds, CEUR Workshop Proceedings, 2026.
- [21] M. Suda, Efficient Neural Clause-Selection Reinforcement, in: *Proceedings of the 30th International Conference on Automated Deduction*, C. Barrett and U. Waldmann, eds, Lecture Notes in Artificial Intelligence, Springer, Berlin, Germany, 2025, pp. 403–422. doi:10.1007/978-3-031-99984-0_22.
- [22] N. Smallbone, Twee: An Equational Theorem Prover (System Description), in: *Proceedings of the 28th International Conference on Automated Deduction*, A. Platzer and G. Sutcliffe, eds, Lecture Notes in Computer Science, Springer, Berlin, Germany, 2021, pp. 602–613.
- [23] N. Murzin, FindProof: Equational Theorem Proving in the Wolfram Language, 2026, Wolfram Institute Technical Note.
- [24] L. Bachmair, N. Dershowitz and D.A. Plaisted, Completion Without Failure, in: *Resolution of Equations in Algebraic Structures*, H. Ait-Kaci and M. Nivat, eds, Academic Press, 1989, pp. 1–30.
- [25] T. Hillenbrand and B. Löchner, The Next Waldmeister Loop, in: *Proceedings of the 18th International Conference on Automated Deduction*, A. Voronkov, ed., Lecture Notes in Artificial Intelligence, Springer, Berlin, Germany, 2002, pp. 486–500.
- [26] T. Kim and A. Pease, SigmaKEE-rs: An Embedded Ontological Reasoning System with Vampire as a Reusable Library Component, 2026, Presented at the 9th Vampire Workshop. <https://adampease.org/Sigma-rs.pdf>.
- [27] A. Pease and S. Schulz, Knowledge Engineering for Large Ontologies with Sigma KEE 3.0, in: *Proceedings of the 7th International Joint Conference on Automated Reasoning*, S. Demri, D. Kapur and C. Weidenbach, eds, Lecture Notes in Artificial Intelligence, Springer, Berlin, Germany, 2014, pp. 519–525.
- [28] A. Pease, *Ontology: A Practical Guide*, Articulate Software Press, 2011.
- [29] K. Hoder and A. Voronkov, Sine Qua Non for Large Theory Reasoning, in: *Proceedings of the 23rd International Conference on Automated Deduction*, V. Sofronie-Stokkermans and N. Bjørner, eds, Lecture Notes in Artificial Intelligence, Springer, Berlin, Germany, 2011, pp. 299–314.
- [30] Q. Liu and Y. Xu, Axiom Selection Over Large Theory Based on New First-order Formula Metrics, *Applied Intelligence* **52**(2) (2022), 1793–1807. doi:10.1007/s10489-021-02469-1.
- [31] O. Roland, Trust What You Prove: Proof Verification in the mrs Ecosystem, Zenodo, 2026. doi:10.5281/zenodo.22066966.
- [32] L. Bachmair and H. Ganzinger, Rewrite-based Equational Theorem Proving with Selection and Simplification, *Journal of Logic and Computation* **4**(3) (1994), 217–247.
- [33] A. Voronkov, AVATAR: The New Architecture for First-Order Theorem Provers, in: *Proceedings of the 26th International Conference on Computer Aided Verification*, A. Biere and R. Bloem, eds, Lecture Notes in Computer Science, 2014, pp. 696–710.
- [34] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleys and F. Pollitt, CaDiCaL 2.0, in: *Proceedings of the 36th International Conference on Computer Aided Verification*, A. Gurfinkel and V. Ganesh, eds, Lecture Notes in Computer Science, Springer, Berlin, Germany, 2024, pp. 133–152.
- [35] G. Sutcliffe, The 12th IJCAR Automated Theorem Proving System Competition - CASC-J12, *AI Communications* **38**(1) (2025), 3–20. doi:10.1177/30504554241305110.
- [36] G. Sutcliffe, Semantic Derivation Verification: Techniques and Implementation, *International Journal on Artificial Intelligence Tools* **15**(6) (2006), 1053–1070. doi:10.1142/S0218213006003119.
- [37] J. Cailler and S. Guilloud, Proceedings of the first Proof Verification Competition (ProoVer 2026), 2026. https://proover-competition.github.io/assets/ProoVer2026_proceedings.pdf.
- [38] G. Sutcliffe, S. Schulz, K. Claessen and A. Van Gelder, Using the TPTP Language for Writing Derivations and Finite Interpretations, in: *Proceedings of the 3rd International Joint Conference on Automated Reasoning*, U. Furbach and N. Shankar, eds, Lecture Notes in Artificial Intelligence, Springer, Berlin, Germany, 2006, pp. 67–81.